

Multi-Class Emotion Detection in Educational Dialogues

By - Ratnam B. Ojha

Main issue we are working on as of now:

1. *Lack of Context-Aware Emotional State Inference in Educational Dialogues:*

Current educational chatbots cannot infer student emotional states from conversational patterns, leading to inappropriate responses when students are frustrated, confused, or losing motivation.

2. No robust NLP models for real-time emotional state classification from educational dialogue context.

3. To fix this, we'll primarily work on NLP and emotion-detection through text analysis.

Overall procedure we're going to follow:

1. Data collection:

A. We use Google's GoEmotion's dataset directly from Hugging Face using "dataset=load_dataset(["google-research-datasets/go_emotions"](#))"

B. There are 28 different labels in the GoEmotion's dataset like:

```
# Get label names from dataset features
label_names = dataset['train'].features['labels'].feature.names
print(label_names)
```

```
['admiration', 'amusement', 'anger', 'annoyance', 'approval', 'caring', 'confusion', 'curiosity', 'desire', 'disappointment', 'disapproval', 'disgust', 'embarrassment', 'excitement', 'fear', 'gratitude', 'grief', 'joy', 'love', 'nervousness', 'optimism', 'pride', 'realization', 'relief', 'remorse', 'sadness', 'surprise', 'neutral']
```

2. Decoding Labels:

We decode the list of label indices into decoded labels and drop the 'labels' and 'id' columns because we don't really need it.

3. Custom Dataset and merging:

Since in the given labels, we don't see the labels such as "Confused", "Frustrated", "Engaged", "Disengaged", we make a custom dataset and then merge it with it with the Go_emotions.csv dataset.

```
df_custom['decoded_labels'] = df_custom['label'].apply(lambda x: [x])
df_custom = df_custom.drop(columns=['label'])
```

```
df_merged = pd.concat([df, df_custom], ignore_index=True)
```

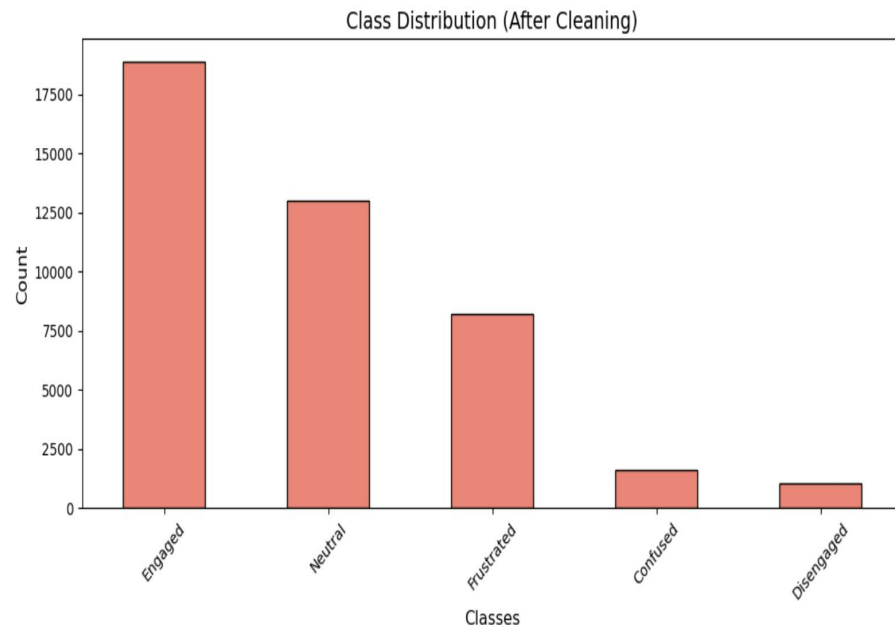
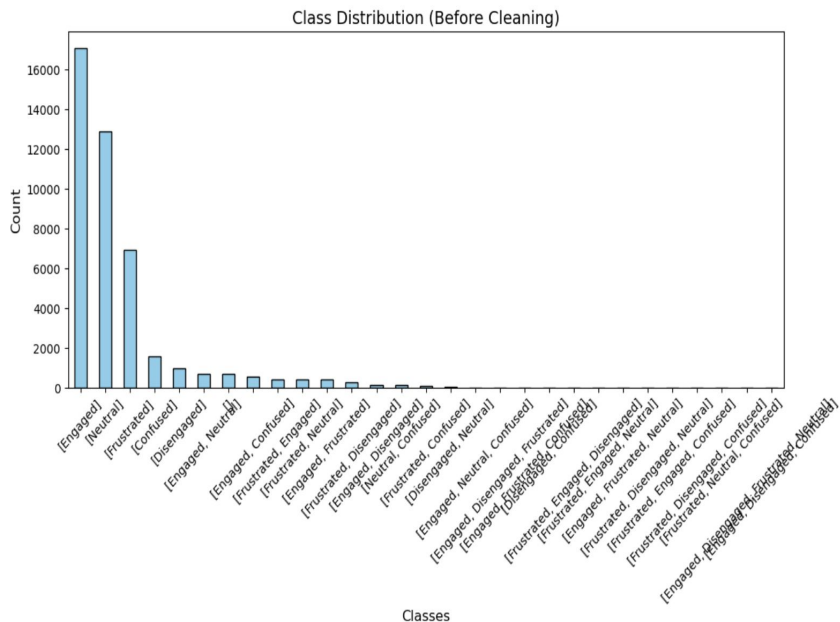
```
print(df_merged.head())
print("Total samples:", len(df_merged))
```

```

      text decoded_labels
0  My favourite food is anything I didn't have to... [neutral]
1  Now if he does off himself, everyone will thin... [neutral]
2  WHY THE FUCK IS BAYLESS ISOING [anger]
3  To make her feel threatened [fear]
4  Dirty Southern Wankers [annoyance]
Total samples: 43441
```

4. Label Cleaning:

Since there are 27 different emotions, which can make our analysis a little complex, we map these 27 emotions into 5 broad categories of emotions: **Confused, Frustrated, Engaged, Disengaged, Neutral**. We do this to tackle problems such as **class imbalance, dataset sparsity, and interpretability issues**.

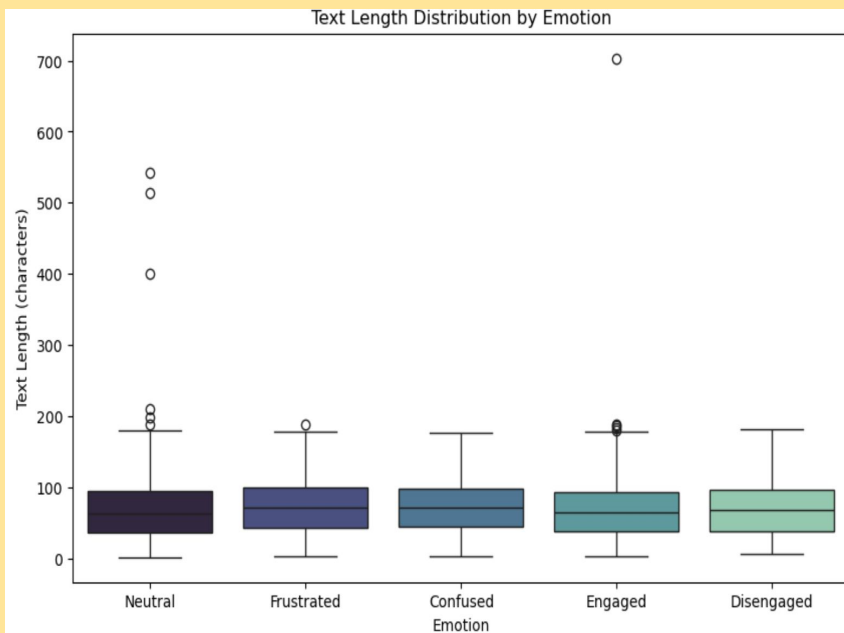


5. Data Cleaning and Preprocessing: [Dealing with Outliers]

We check for outliers on the basis of text length and then remove these outliers. Very short texts lack context (e.g., “ok”, “hmm”) → not useful for emotion detection.

Very long texts introduce noise and skew TF-IDF/embeddings.

Removing such outliers ensures a **cleaner feature space, reduces noise, and improves model generalization.**



```
import pandas as pd
```

```
df_merged['text_len'] = df_merged['text'].apply(len)
```

```
Q1 = df_merged['text_len'].quantile(0.25)
```

```
Q3 = df_merged['text_len'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
print("Q1:", Q1, "Q3:", Q3, "IQR:", IQR)
```

```
print("Lower Bound:", lower_bound, "Upper Bound:", upper_bound)
```

```
/iim_research/chatbot_model.html
```

chatbot_model

```
df_no_outliers = df_merged[(df_merged['text_len'] >= lower_bound) &  
                           (df_merged['text_len'] <= upper_bound)]
```

```
print("Original dataset size:", df_merged.shape[0])
```

```
print("After outlier removal:", df_no_outliers.shape[0])
```

```
Q1: 38.0 Q3: 96.0 IQR: 58.0
```

```
Lower Bound: -49.0 Upper Bound: 183.0
```

```
Original dataset size: 43441
```

```
After outlier removal: 43432
```

6. Data preprocessing: [Dealing with *Text standardization* and *Lemmatization* using SpaCy]

```
import re

def clean_text(text):
    text = text.lower()                # lowercase
    text = re.sub(r"http\S+|www\S+", " URL ", text)    # replace urls
    text = re.sub(r"\d+", " NUM ", text)              # replace numb
    text = re.sub(r"^[^\w\s]", "", text)              # remove punct
    text = re.sub(r"\s+", " ", text).strip()          # remove extra
    return text

df_no_outliers = df_no_outliers.copy()
df_no_outliers['clean_text'] = df_no_outliers['text'].apply(clean_text)
```

- We download a small pre-trained model(sm) from spaCy cli.
- We disable parser and ner(named entity recognition) because Parser helps find grammatical structure in the text and NER helps in recognizing names, places, dates etc. which we don't primarily need for our task.
- Primary focus is on tokenization, lemmatization, and POS tagging.

- Text is converted into a clean, consistent form.
- This reduces noise in the dataset (e.g., "123!!!" and "123" both become "NUM")
- Models don't inherently understand punctuation, case, or URLs. Cleaning ensures that the algorithm focuses on patterns in language.

```
import spacy

nlp = spacy.load("en_core_web_sm", disable=["parser", "ner"])

def lemmatize_text(text):
    doc = nlp(text)
    return " ".join([token.lemma_ for token in doc])

df_no_outliers['lemmatized_text'] = df_no_outliers['clean_text'].apply(lemmatize_text)

text = "cats and dogs are running"
lemmatize_text(text)
```

'cat and dog be run'

7. Feature addition: [adding columns for word_count and char_count]

```
df_no_outliers['word_count'] = df_no_outliers['lemmatized_text'].apply(lambda x: len(x.split()))
df_no_outliers['char_count'] = df_no_outliers['lemmatized_text'].apply(len)
```

- We add a new column for “word_count” => This function takes the text from the column `lemmatized_text`. Splits it into words (`x.split()`) and counts the number of words.
- A new column “char_count”, counts the total **number of characters** (including spaces) in the text.
- **Word count:** Longer sentences often indicate *Engaged* or *Frustrated* states, while very short ones lean toward *Neutral*.
Char count: A complementary signal to detect unusually short or long responses.

8. Feature addition: [adding columns for polarity scores]

```
from nltk.sentiment import SentimentIntensityAnalyzer
import nltk
nltk.download('vader_lexicon')

sia = SentimentIntensityAnalyzer()

def get_sentiment(text):
    scores = sia.polarity_scores(text)
    return scores['neg'], scores['neu'], scores['pos'], scores['compound']

df_no_outliers[['sent_neg', 'sent_neu', 'sent_pos', 'sent_compound']] = df_no_outliers['lemmatized_text'].apply(
    lambda x: pd.Series(get_sentiment(x))
)
```

- We import NLTK's VADER sentiment analyzer and download pre-trained lexicon.
- SentimentIntensityAnalyzer analyzes text polarity scores. Analyzes text polarity scores
- Returns 4 metrics: negative, neutral, positive, compound (-1 to +1) and new columns are created.

9. Feature addition: [adding TF-IDF feature]

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import FeatureUnion

# Word-level TF-IDF (existing)
word_tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

# Char-level TF-IDF (new)
char_tfidf = TfidfVectorizer(analyzer="char", ngram_range=(3,6), max_features=3000)

# Combine both
text_features = FeatureUnion([
    ("word_tfidf", word_tfidf),
    ("char_tfidf", char_tfidf)
])
```

- Machine Learning models **don't understand raw text**. They can only interpret **numerical values**. We use TF-IDF feature to convert text to numbers.
- “Word_tfidf” analyzes **whole words** and **word pairs** and creates 5000 most important word-based features.
- “Char_tfidf” analyzes **character sequences** of 3-6 characters and creates 3000 most important character-based features.
- We use “**FeatureUnion**” function to run these two tfidf functions simultaneously.

10. Feature addition: [adding POS(Part-of-Speech) tagging feature]

```
import spacy
nlp = spacy.load("en_core_web_sm", disable=["parser", "ner"])

def get_pos_counts(text):
    doc = nlp(text)
    noun_count = sum(1 for token in doc if token.pos_ == "NOUN")
    verb_count = sum(1 for token in doc if token.pos_ == "VERB")
    adj_count = sum(1 for token in doc if token.pos_ == "ADJ")
    pron_count = sum(1 for token in doc if token.pos_ == "PRON")
    return noun_count, verb_count, adj_count, pron_count

df_no_outliers[["noun_count", "verb_count", "adj_count", "pron_count"]] = (
    df_no_outliers["lemmatized_text"].apply(lambda x: pd.Series(get_pos_counts(x)))
)
```

- **POS Tagging:** SpaCy identifies grammatical roles of each word.
- This function counts nouns, verbs, adjectives, pronouns in a given sentence.
- Emotions often correlate with certain POS usage (e.g., adjectives in “*I’m frustrated*”). This helps in adding linguistic structure features beyond raw text.
- New columns with noun_count, verb_count, adj_count and pronoun_count are added.

11. Model Training:

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

# Features: text + numeric
X = df_no_outliers[[
    'clean_text', 'text_len', 'word_count', 'char_count',
    'sent_neg', 'sent_neu', 'sent_pos', 'sent_compound',
    'noun_count', 'verb_count', 'adj_count', 'pron_count'
]]
y = df_no_outliers['final_label']

# Preprocessing for text
tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))

# Preprocessing for numeric features
scaler = StandardScaler()

# Combine text + ALL numeric features
preprocessor = ColumnTransformer(
    transformers=[
        ('text', tfidf, 'clean_text'),
        ('num', scaler, [
            'text_len', 'word_count', 'char_count',
            'sent_neg', 'sent_neu', 'sent_pos', 'sent_compound',
            'noun_count', 'verb_count', 'adj_count', 'pron_count'
        ])
    ]
)
```

```
# Build full pipeline
clf = Pipeline(steps=[
    ('preprocess', preprocessor),
    ('classifier', model)
])
```

```
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, f1_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier
from xgboost import XGBClassifier
```

```
import numpy as np
```

```
# Step 1: Clean labels properly
```

```
y_clean = y.apply(lambda lbl: lbl[0] if isinstance(lbl, list) and len(lbl) > 0 else str(lbl))
```

```
# Step 2: Drop rows where label is ""
```

```
mask = y_clean != ""
```

```
X = X[mask]
```

```
y_clean = y_clean[mask]
```

```
# Step 3: Check class counts again
```

```
class_counts = y_clean.value_counts()
```

```
print("Class distribution after cleaning:\n", class_counts)
```

```
# Step 4: Merge rare classes (<2 samples) into "Other"
```

```
y_balanced = y_clean.apply(lambda lbl: lbl if class_counts.get(lbl, 0) >= 2 else "Other")
```

- Now, we split the data into X and y set. X has all the numerical+text features and y has the feature that has to be predicted i.e, "final_label".
- We drop the rows where the label is "" i.e, empty rows. After this, we check the Class Distribution after data cleaning.

```
from sklearn.ensemble import AdaBoostClassifier

X_train, X_test, y_train, y_test = train_test_split(
    X, y_balanced, test_size=0.3, random_state=42, stratify=y_balanced
)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "SVM": SVC(kernel="linear", probability=True),
    "Random Forest": RandomForestClassifier(n_estimators=200, random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=200, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=200, random_state=42),
    "MLP": MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)
}
```

- We split the X and y into training and testing set. 70% of the data is used as training set and 30% of the data is used as testing set.
- We use 6 different models on our dataset - LogReg, SVM, Random Forest, KNN, Gradient Boosting, AdaBoost, MLP.

12. Choosing Metrics and Insights we get:

```
import pandas as pd
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.pipeline import Pipeline
from sklearn.ensemble import AdaBoostClassifier

for name, model in models.items():
    clf = Pipeline(steps=[
        ('preprocess', preprocessor),
        ('classifier', model)
    ])

    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred, average="weighted")
    precision = precision_score(y_test, y_pred, average="macro")
    recall = recall_score(y_test, y_pred, average="macro")

    auc = None
    if hasattr(clf, "predict_proba"):
        auc = roc_auc_score(y_test, y_prob, multi_class="ovr")
    except Exception:
        pass

    print(f"\n=== {name} ===")
    print("Accuracy:", round(acc, 4))
    print("Weighted F1:", round(f1, 4))
    print("Macro Precision:", round(precision, 4))
    print("Macro Recall:", round(recall, 4))
    if auc is not None:
        print("AUC-ROC:", round(auc, 4))
```

Choosing metrics:

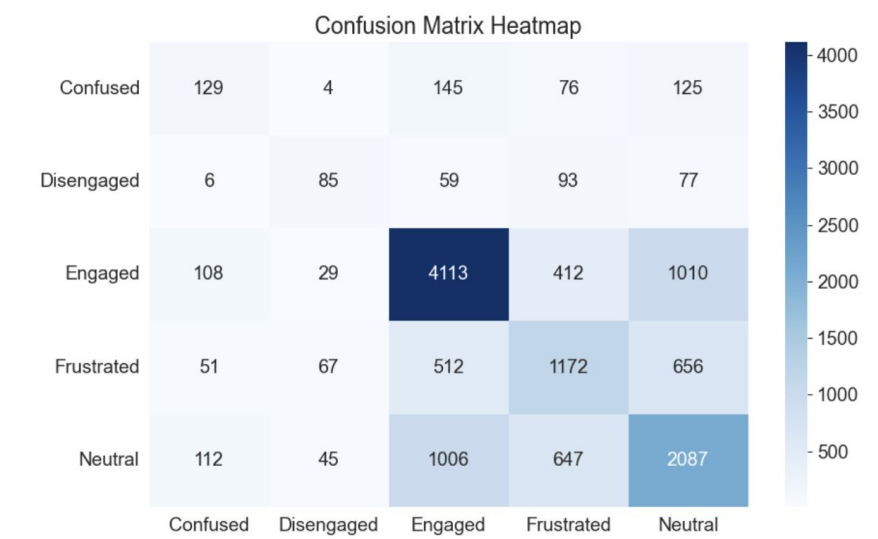
- **Accuracy:**
Measures overall correctness percentage across all predictions.
Simple baseline but misleading with imbalanced datasets
- **Weighted F1:**
F1 score adjusted for class distribution in the dataset. It is better at reflecting real-world performance than raw accuracy.
- **Macro Precision:**
Average precision across all classes without weighting.
Ensures minority classes aren't ignored by the model
- **Micro Recall:**
Average recall across all classes equally weighted. Prevents model bias toward majority class predictions
- **AUC- ROC:**
Measures model's ranking ability independent of threshold
Best indicator of model's confidence calibration quality.

Insights:

- **SVM** slightly outperforms **Logistic Regression** in terms of both *Weighted F1* (0.6328 vs 0.629) and *Macro Precision/Recall*, making it the strongest linear model. Logistic Regression is very close though, showing both models capture patterns similarly.
- **Random Forest** lags behind **SVM** and **Logistic Regression** in *Accuracy* (0.6169) and *Weighted F1* (0.6021), even though its *Macro Precision* (0.6423) is higher. This means RF predicts some classes very well but misses others (low Recall = 0.4088).
- **Gradient Boosting** performs slightly better than **Random Forest** in terms of *Weighted F1* (0.6203) and *Recall* (0.4719), showing boosting handles imbalance better than bagging in this dataset.
- **KNN** struggles with both *Accuracy* (0.5343) and *Macro Precision/Recall*, suggesting it's not capturing class boundaries well. **AdaBoost** also underperforms with *Accuracy* = 0.5728 and *Weighted F1* = 0.5567, showing it's too simple for this task.
- AUC-ROC scores are highest for *Logistic Regression* (0.8529) and *SVM* (0.8508), confirming that linear classifiers separate classes better than tree-based ones here.
- The low *Macro Recall values* (0.30–0.43) across models indicate difficulty in detecting minority classes (Confused, Disengaged, Frustrated). Most models focus on the majority classes (Engaged, Neutral).
- Techniques like class re-weighting, oversampling (SMOTE), or using deep learning transformers (e.g., BERT) could improve minority class detection.

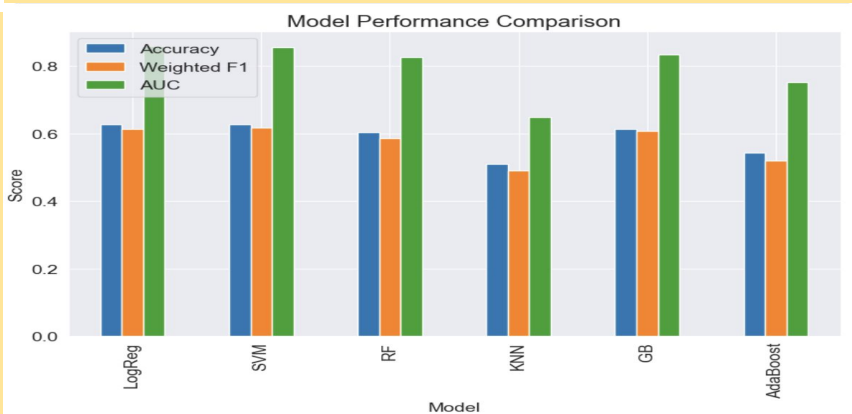
13. Conclusion after model training and our next approach:

Using the trained model, we generate a confusion matrix. This will give us an insight into where our model fails or succeeds.



- The model demonstrates moderate performance with a strong bias towards accurately identifying the **Engaged state**, while struggling significantly with negative/passive states (**Confused , Disengaged**).
- Our model is great at detecting **Engaged** states (4,220 correct predictions)
- Clear dominance on the primary diagonal for positive engagement.
- Poor identification of **Confused states** (only 115 correct). Extremely weak **Disengaged detection** (only 116 correct). High misclassification rates between negative and positive states.

After looking at these graphs, we can conclude that our model is giving the best performance for the SVM algorithm.



14. Testing our model and solving the problem:

Input: I understand everything and feel confident.
Predicted Emotion: Neutral

Input: This topic is very confusing, I don't get it.
Predicted Emotion: Frustrated

Input: I don't feel like paying attention anymore.
Predicted Emotion: Neutral

Input: I feel stuck and frustrated with the exercises.
Predicted Emotion: Neutral

Input: I am just listening without much reaction.
Predicted Emotion: Neutral

We see that most of the inputs by the students are biased towards "**Engaged**", "**Frustrated**" as results. This is because of the biasness towards the majority classes.

We can solve this problem by:

- Applying **SMOTE** [*Synthetic Minority Oversampling Technique*]
- Using **BERT** [*Bidirectional Encoder Representation of Transformers*]

15. Applying SMOTE:

- Since SMOTE cannot be performed on "Labeled" data, we need to convert the labels into numerical form.
- To do this, we use LabelEncoder which converts *categorical data* to *numerical data*.
- Now, every category will correspond to a certain index like: 0 → 'Confused' 1 → 'Disengaged' 2 → 'Engaged' 3 → 'Frustrated' 4 → 'Neutral'.

```
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, accuracy_score
from sklearn.model_selection import GridSearchCV
from sklearn.preprocessing import LabelEncoder

# Handle lists and empty lists
y_flat = y.apply(lambda x: x[0] if isinstance(x, list) and len(x) > 0 else np.nan)

# Drop rows with missing labels
y_flat = y_flat.dropna()

# Re-align X with y
X = X.loc[y_flat.index]

# Encode
le = LabelEncoder()
y_encoded = le.fit_transform(y_flat)

# Train-test (variable) X_test: Any
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.3, random_state=42, stratify=y_encoded
)

# Transform features
X_train_transformed = preprocessor.fit_transform(X_train)

# Apply SMOTE
smote = SMOTE(random_state=42, k_neighbors=5)
X_resampled, y_resampled = smote.fit_resample(X_train_transformed, y_train)
```

16. SMOTE Results:

KNN hyperparameter tuning:

```
param_grid = {
    'n_neighbors': [3, 5, 7, 9, 11],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

knn = KNeighborsClassifier()
grid_search = GridSearchCV(
    estimator=knn,
    param_grid=param_grid,
    cv=5,
    n_jobs=-1,
    scoring='f1_macro'
)

# Fit on SMOTE data
grid_search.fit(X_resampled, y_resampled)

print("Best Parameters:", grid_search.best_params_)
print("Best CV Score:", grid_search.best_score_)

best_knn = grid_search.best_estimator_

X_test_transformed = preprocessor.transform(X_test)
y_pred = best_knn.predict(X_test_transformed)

print("\nAccuracy on Test Set:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

Best Parameters: {'metric': 'euclidean', 'n_neighbors': 3, 'weights': 'distance'}
Best CV Score: 0.7641978520946939

Accuracy on Test Set: 0.337361609231249

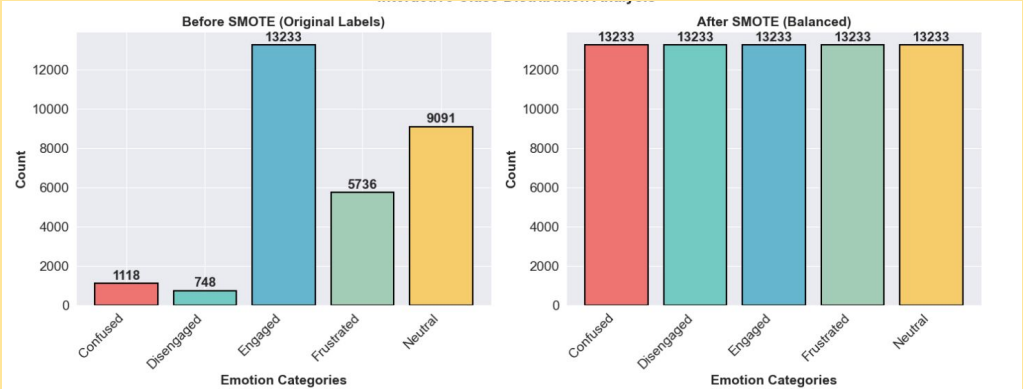
Classification Report:
      precision    recall  f1-score   support

0         0.05      0.32      0.09         479
1         0.06      0.28      0.10         320
2         0.78      0.31      0.44        5672
3         0.31      0.39      0.34        2458
4         0.42      0.36      0.39        3897

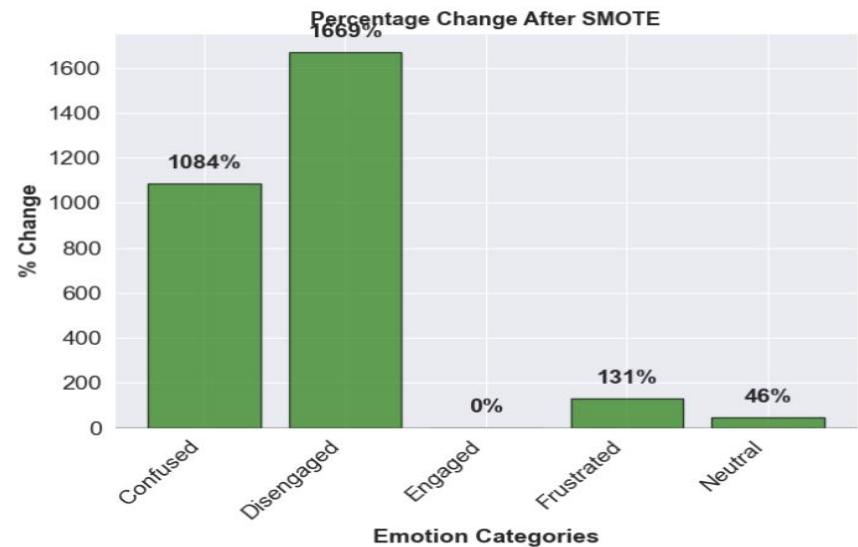
accuracy          0.33
macro avg         0.33
weighted avg      0.34
```

- GridSearchCV tests 30 combinations of KNN parameters using 5-fold cross-validation.
- Optimizes n_neighbors (3-11), weights (uniform/distance), and distance metrics.
- Odd neighbor counts prevent voting ties, distance weighting gives closer points more influence.
- Multiple distance metrics handle different data distributions effectively.
- Cross-validation prevents overfitting to specific data splits.

Class distribution before and after SMOTE



Changes in Class Distribution after SMOTE:



Class Mapping Reference Table

Numeric	Original Label	Before SMOTE	After SMOTE
Class 0	Confused	1,118	13,233
Class 1	Disengaged	748	13,233
Class 2	Engaged	13,233	13,233
Class 3	Frustrated	5,736	13,233
Class 4	Neutral	9,091	13,233

SUMMARY

Class Distribution Summary:

Class 0 ('Confused'): 1,118 → 13,233

Class 1 ('Disengaged'): 748 → 13,233

Class 2 ('Engaged'): 13,233 → 13,233

Class 3 ('Frustrated'): 5,736 → 13,233

Class 4 ('Neutral'): 9,091 → 13,233

Why SMOTE isn't the best approach to fix our problem:

- **Poor Generalization on Test Set :**

1. Although GridSearchCV reported a relatively high cross-validation score (CVScore ≈ 0.76), the test set accuracy dropped sharply to ≈ 0.34 .
2. This indicates that the model overfits to the synthetic data generated by SMOTE but failed to generalize to real-world unseen examples.

- **Class-wise Performance Degradation :**

1. Minority classes (0 and 1) - Precision and recall are *extremely low* (Precision: 0.05–0.06, Recall: 0.28–0.32). F1-scores are near 0.09–0.10, meaning the model *barely recognizes* these intents correctly.
2. Majority classes (2, 3, 4) - Class 2 has high precision (0.78) but very poor recall (0.31), suggesting the model predicts *this class too selectively*.
3. Classes 3 and 4 are moderately recognized, but performance is still unsatisfactory (F1-scores ~ 0.34 –0.39).

- **Mismatch between SMOTE and Text data :**

1. SMOTE works by interpolating between feature vectors.
2. For text embeddings (TF-IDF or similar), this interpolation produces synthetic samples that don't correspond to meaningful sentences. This is why the classifier picked up misleading patterns, hurting test performance.

- **Macro and Weighted score performance comparison :**

1. Macro Avg F1 = 0.27, showing the model fails to treat all classes equally.
2. Weighted Avg F1 = 0.38, slightly better because majority classes dominate, but still far below acceptable performance for a chatbot.